

UNIVERSITÀ DEGLI STUDI DI MESSINA
CORSO DI LAUREA IN INFORMATICA

Flight Reservation System

Massimiliano Spartà e Simone Adamo

Matricola 566093 e 567317

ANNO ACCADEMICO 2025–2026

INDICE

1	Neo4 e Cassandra	2
1.1	Introduzione	2
1.2	Problema affrontato	2
1.3	Soluzione DBMS considerata	3
1.3.1	Neo4j	3
1.3.2	Apache Cassandra	4
1.4	Progettazione	5
1.4.1	Schema Neo4j	5
1.5	Esempio per Neo4j	6
1.5.1	Schema Cassandra	7
1.6	Implementazione	8
1.6.1	Generazione dei dati	8
1.6.2	Inserimento	8
1.7	Query di benchmark	10
1.7.1	Automazione del benchmark	11
1.8	Esperimenti	12
1.8.1	Configurazione sperimentale	12
1.9	Risultati	12
1.10	Analisi dei grafici	14
1.11	Cold vs Warm cache	16
1.12	Conclusioni	17
1.13	Appendice: Risultati con container Cassandra non ricreato .	18

1 NEO4 E CASSANDRA

1.1 Introduzione

Il presente progetto si inserisce nell'ambito del modulo NoSQL del corso di Basi di Dati. L'obiettivo è confrontare due sistemi di gestione di basi di dati appartenenti a paradigmi diversi, un database a grafo (**Neo4j**) e un database wide-column (**Apache Cassandra**), sul medesimo caso di studio, al fine di evidenziare i diversi trade-off in termini di prestazioni, scalabilità e flessibilità di interrogazione.

Il caso di studio scelto è il *Flight Reservation System* (traccia 2.B), un dominio fortemente relazionale caratterizzato da entità interconnesse (voli, passeggeri, prenotazioni, compagnie aeree, aeroporti, posti) che si presta bene a mettere in luce le differenze tra un modello nativamente grafo e un modello che richiede una progettazione *query-first* con denormalizzazione dei dati.

Gli esperimenti sono stati condotti a parità di condizioni hardware e software, utilizzando container Docker con risorse equivalenti (2 GB di heap, stessa CPU), e considerando quattro dataset di dimensione crescente (25%, 50%, 75%, 100%) e quattro query di complessità crescente. Per ogni configurazione sono state eseguite 31 iterazioni, misurando il tempo della prima esecuzione (*cold start*) e la media delle successive 30 (*warm cache*), con relativi intervalli di confidenza al 95%.

1.2 Problema affrontato

Il sistema da modellare gestisce le prenotazioni di voli aerei in un aeroporto municipale. Le entità principali individuate sono:

- **Flight** (id, number, departure_date, arrival_date, status);
- **Passenger** (id, name, first_name, passport, email);
- **Reservation** (id, reservation_date, status, total_price);
- **Airline** (id, name, code, country);
- **Airport** (id, code, name, city, country);

- **Seat** (id, number, class, status).

Le relazioni semantiche sono le seguenti:

- un passeggero *riserva* un volo (tramite Reservation);
- un volo *parte da* un aeroporto e *arriva a* un aeroporto;
- un volo è *operato da* una compagnia aerea;
- un passeggero *occupa* un posto (tramite Reservation).

Per migliorare la completezza del caso di studio del modello entità/re-lazione è stato ampliato rispetto alla traccia originale; in particolare, l'entità Seat è stata introdotta per rendere più realistico il dominio e per aumentare il numero di relazioni coinvolte nelle query più complesse.

Sono stati generati quattro dataset con Faker, ottenuti per estrazione a catena dal dataset al 100% ($25\% \subset 50\% \subset 75\% \subset 100\%$), come richiesto dalla traccia:

- **25%:** 1.000 voli, 38.019 passeggeri coinvolti, 150.000 posti, 120.000 prenotazioni;
- **50%:** 2.000 voli, 39.904 passeggeri coinvolti, 300.000 posti, 240.000 prenotazioni;
- **75%:** 3.000 voli, 39.996 passeggeri coinvolti, 450.000 posti, 360.000 prenotazioni;
- **100%:** 4.000 voli, 40.000 passeggeri, 600.000 posti, 480.000 prenotazioni.

Il numero di passeggeri non scala linearmente con la percentuale di voli: il pool di 40.000 passeggeri viene ripescato con ripetizione da centinaia di migliaia di prenotazioni, per cui anche il sottoinsieme più piccolo (25% dei voli) coinvolge già la quasi totalità del pool. Voli, posti e prenotazioni, al contrario, scalano esattamente con la percentuale richiesta, poiché derivano direttamente dal sottoinsieme di voli estratto.

1.3 Soluzione DBMS considerata

1.3.1 Neo4j

Neo4j è un database a grafo nativo che modella i dati come nodi (etichette) e relazioni (tipi direzionali) con proprietà associate.

È particolarmente efficace quando le interrogazioni richiedono traversali multi-hop tra entità correlate. Nel nostro progetto è stato utilizzato Neo4j 5.24 in container Docker. Il modello dati adottato è un *property graph* in cui:

- ogni entità diventa un nodo con una label (:Flight, :Passenger, ...);
- ogni relazione dello schema ER diventa un arco tipizzato (:MAKES, :FOR_FLIGHT, :DEPARTS_FROM, ...);
- Reservation è modellata come nodo intermedio (relazione reificata) poiché lega tre entità (passeggero, volo, posto).

Sono stati definiti vincoli di unicità (CREATE CONSTRAINT) sui campi chiave e indici secondari sui campi utilizzati nei filtri (status, departure_date).

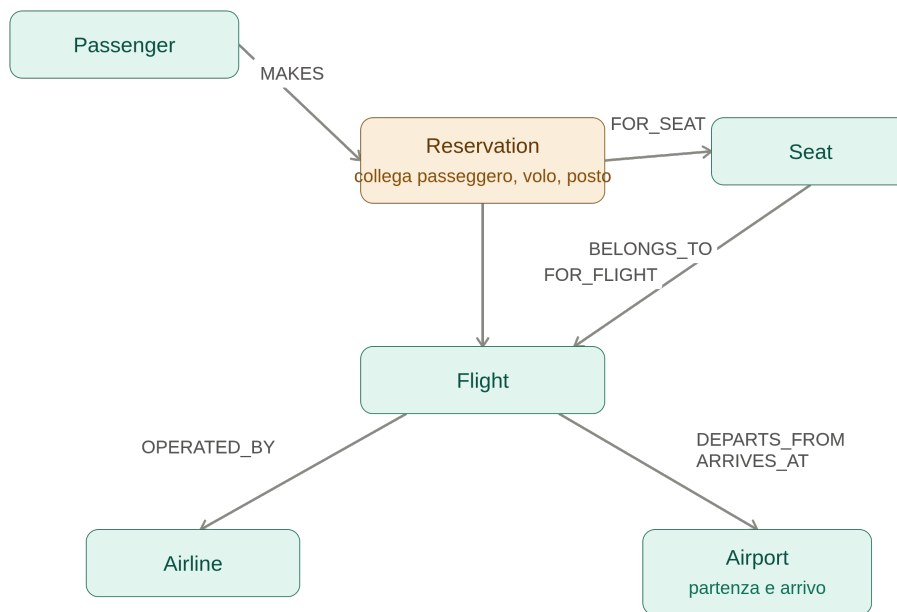


Figura 1.1: Modello a grafo di Neo4j

1.3.2 Apache Cassandra

Cassandra è un database distribuito di tipo wide-column store, progettato per scalabilità orizzontale e alta disponibilità. A differenza di Neo4j, Cassandra non supporta nativamente operazioni di JOIN: ogni tabella deve essere progettata sulla base di una specifica query di accesso (*query-first*

design). Per il caso di studio è stato creato il keyspace `flights_ks` con quattro tabelle denormalizzate, ciascuna ottimizzata per una delle quattro query di benchmark:

- `flights_by_status` (Q1): ricerca voli per stato;
- `reservations_by_passenger` (Q2): prenotazioni di un passeggero con dettagli volo annidati;
- `reservations_by_airline_date` (Q3): prenotazioni per compagnia in un intervallo di date;
- `reservations_by_airline_airport` (Q4): aggregazione per compagnia, aeroporto di partenza e periodo.

La denormalizzazione comporta una duplicazione dei dati (i dettagli di volo, passeggero e posto sono replicati nelle tabelle Q2, Q3 e Q4), ma garantisce tempi di lettura costanti indipendentemente dalla complessità relazionale.

1.4 Progettazione

1.4.1 Schema Neo4j

Il grafo è stato strutturato con i seguenti vincoli:

```
1 CREATE CONSTRAINT flight_id IF NOT EXISTS
2   FOR (f:Flight) REQUIRE f.id IS UNIQUE;
3 CREATE CONSTRAINT passenger_id IF NOT EXISTS
4   FOR (p:Passenger) REQUIRE p.id IS UNIQUE;
5 CREATE CONSTRAINT airport_code IF NOT EXISTS
6   FOR (a:Airport) REQUIRE a.code IS UNIQUE;
7 CREATE CONSTRAINT airline_code IF NOT EXISTS
8   FOR (a:Airline) REQUIRE a.code IS UNIQUE;
9 CREATE CONSTRAINT seat_id IF NOT EXISTS
10  FOR (s:Seat) REQUIRE s.id IS UNIQUE;
11 CREATE CONSTRAINT reservation_id IF NOT EXISTS
12  FOR (r:Reservation) REQUIRE r.id IS UNIQUE;
13
14 CREATE INDEX flight_status IF NOT EXISTS
15   FOR (f:Flight) ON (f.status);
16 CREATE INDEX flight_departure_date IF NOT EXISTS
17   FOR (f:Flight) ON (f.departure_date);
```

Listing 1.1: Vincoli e indici Neo4j

Le relazioni principali sono:

- (:Passenger)-[:MAKES]->(:Reservation)
- (:Reservation)-[:FOR_FLIGHT]->(:Flight)
- (:Reservation)-[:FOR_SEAT]->(:Seat)
- (:Flight)-[:DEPARTS_FROM]->(:Airport)
- (:Flight)-[:ARRIVES_AT]->(:Airport)
- (:Flight)-[:OPERATED_BY]->(:Airline)
- (:Seat)-[:BELONGS_TO]->(:Flight)

1.5 Esempio per Neo4j

Mario Rossi (passeggero id 1) prenota il volo AZ100 (Roma FCO → Parigi CDG, 10 giugno), operato da ITA Airways, posto 12A. Questa prenotazione ha il numero 501, costa 250€, è confermata.

Ecco esattamente come questi dati vivono nel database Neo4j:

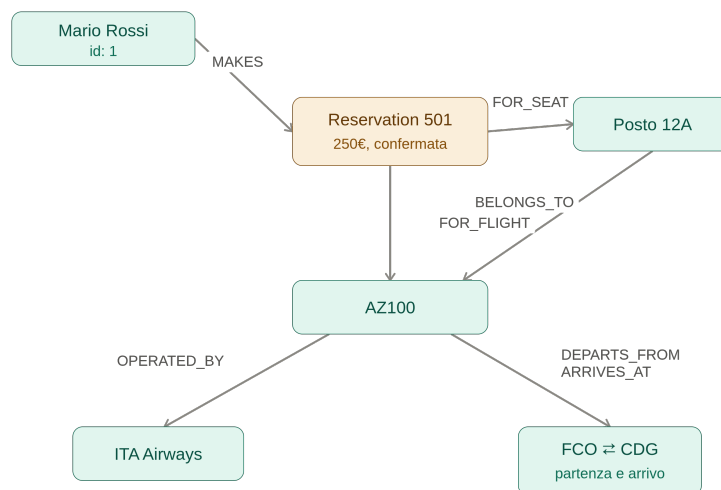


Figura 1.2: Esempio passeggero Mario Rossi

1.5.1 Schema Cassandra

Il keyspace è stato definito con fattore di replicazione 1 (ambiente single-node):

```
1 CREATE KEYSPACE IF NOT EXISTS flights_ks
2   WITH replication = {'class': 'SimpleStrategy', '
   replication_factor': 1};
3
4 CREATE TABLE flights_by_status (
5   status text, flight_id int, number text,
6   departure_date timestamp, arrival_date timestamp,
7   departure_airport text, arrival_airport text,
8   airline_code text,
9   PRIMARY KEY (status, flight_id)
10 );
11
12 CREATE TABLE reservations_by_passenger (
13   passenger_id int, reservation_id int,
14   flight_number text, departure_date timestamp,
15   arrival_date timestamp, flight_status text,
16   reservation_status text, total_price decimal,
17   PRIMARY KEY (passenger_id, reservation_id)
18 );
19
20 CREATE TABLE reservations_by_airline_date (
21   airline_code text, departure_date timestamp,
22   reservation_id int, passenger_name text,
23   passenger_email text, flight_number text,
24   seat_number text, seat_class text, total_price decimal
25   ,
26   PRIMARY KEY (airline_code, departure_date,
27   reservation_id)
28 ) WITH CLUSTERING ORDER BY (departure_date DESC);
29
30 CREATE TABLE reservations_by_airline_airport (
31   airline_code text, departure_airport text,
32   departure_date timestamp, reservation_id int,
33   total_price decimal,
34   PRIMARY KEY ((airline_code, departure_airport),
35   departure_date, reservation_id)
36 ) WITH CLUSTERING ORDER BY (departure_date DESC);
```

Listing 1.2: Schema CQL Cassandra

La tabella `reservations_by_airline_airport` include `departure_date` come clustering column per supportare efficientemente il filtro per intervallo temporale nella Q4.

1.6 Implementazione

1.6.1 Generazione dei dati

I dataset sono stati generati tramite la libreria Python `Faker` con seme fisso (`seed=42`) per garantire riproducibilità. Per ogni compagnia aerea è stato definito un *hub* aeroportuale (il primo aeroporto della lista) in modo che il 50% dei voli parta da tale hub, garantendo dati significativi per la Q4.

Il dataset al 100% (voli, posti, prenotazioni, pool di passeggeri) viene generato una sola volta. I tre dataset più piccoli sono ottenuti per **campionamento a catena**: il 75% è estratto come sottoinsieme del 100%, il 50% come sottoinsieme del 75% già estratto, il 25% come sottoinsieme del 50%. Questo garantisce un annidamento reale ($25\% \subset 50\% \subset 75\% \subset 100\%$), coerente con quanto richiesto dalla traccia (*“da questo dataset occorrerà estrapolare un sottoinsieme”*), anziché quattro popolazioni generate indipendentemente e solo statisticamente simili tra loro. Posti e prenotazioni discendono a cascata dai voli selezionati; i passeggeri inclusi in ciascun sottoinsieme sono quelli effettivamente referenziati da almeno una prenotazione in quel sottoinsieme.

1.6.2 Inserimento

L’inserimento in Neo4j utilizza il driver ufficiale con il costrutto `UNWIND` per operazioni batch, molto più efficiente rispetto a una transazione per riga:

```
1 session.run("""
2     UNWIND $batch as row
3     MERGE (f:Flight {id: toInteger(row.id)})
4     SET f.number = row.number,
5         f.departure_date = datetime(row.departure_date),
6         f.arrival_date = datetime(row.arrival_date),
7         f.status = row.status
8     WITH f, row
9     MATCH (dep:Airport {code: row.departure_airport})
10    MATCH (arr:Airport {code: row.arrival_airport})
11    MATCH (al:Airline {code: row.airline_code})
12    MERGE (f)-[:DEPARTS_FROM]->(dep)
13    MERGE (f)-[:ARRIVES_AT]->(arr)
```

```

14     MERGE (f)-[:OPERATED_BY]->(a1)
15     """ , batch=batch[i:i+1000])

```

Listing 1.3: Inserimento batch in Neo4j

Per Cassandra si sono utilizzati *prepared statement* e `execute_concurrent` in chunk di 1.000 righe per bilanciare throughput e memoria:

```

1 ps = session.prepare("""
2     INSERT INTO flights_by_status
3     (status, flight_id, number, departure_date, ...)
4     VALUES (?, ?, ?, ?, ...)
5     """)
6 execute_concurrent(session, batch[i:i+1000])

```

Listing 1.4: Inserimento concorrente in Cassandra

Prima di ogni inserimento, entrambi i database vengono ripuliti per garantire che ogni dataset sia confrontato in isolamento. Per Cassandra è sufficiente un `TRUNCATE` sulle singole tabelle, operazione economica indipendentemente dal numero di righe perché non richiede una scansione riga per riga.

Per Neo4j la pulizia si è rivelata meno banale del previsto: un semplice `MATCH (n) DETACH DELETE n` su un grafo di centinaia di migliaia di nodi e milioni di relazioni tenta di cancellare tutto in un'unica transazione, che satura la memoria della JVM (`Neo.TransientError.General.MemoryPoolOutOfMemoryError`), anche suddividendo la cancellazione a lotti con `CALL {...} IN TRANSACTIONS OF 10000 ROWS`. La soluzione adottata è stata di aggirare il problema a livello di infrastruttura: prima di ogni dataset, il container Neo4j viene interamente ricreato (container e volume dati rimossi e ricreati da zero), garantendo un grafo vuoto senza dover eseguire alcuna query di cancellazione su grandi volumi di dati.

Cassandra non ha mostrato lo stesso limite di memoria, per cui un semplice `TRUNCATE` sarebbe stato sufficiente. Per rendere simmetriche le condizioni sperimentali tra i due DBMS, anche il container Cassandra viene però ricreato interamente prima di ogni dataset, con lo stesso meccanismo usato per Neo4j (container e volume rimossi, schema riapplicato da zero). In questo modo entrambi i sistemi ripartono da uno stato realmente *freddo* a ogni dataset, e il confronto sui tempi di prima esecuzione (Sezione 1.11) risulta equo tra i due DBMS.

1.7 Query di benchmark

Le quattro query sono state definite con complessità crescente in termini di entità coinvolte:

Q1 — 1 entità, filtro semplice

```
1 MATCH (f:Flight {status: $status})
2 MATCH (f)-[:DEPARTS_FROM]->(dep:Airport)
3 MATCH (f)-[:ARRIVES_AT]->(arr:Airport)
4 MATCH (f)-[:OPERATED_BY]->(al:Airline)
5 RETURN f.id, f.number, f.departure_date, f.arrival_date,
6        f.status, dep.code, arr.code, al.code
```

Listing 1.5: Q1 Neo4j

```
1 SELECT * FROM flights_by_status WHERE status = ?
```

Listing 1.6: Q1 Cassandra

Una prima versione di questa query lato Neo4j restituiva solo `count(f)`, uno scalare, contro il `SELECT *` di Cassandra che restituisce tutte le righe con tutti i campi: un confronto non valido, perché il contenuto informativo restituito non era lo stesso nei due DBMS, requisito esplicito della traccia. La query corretta fa attraversare a Neo4j le stesse relazioni necessarie per restituire, per ogni volo che soddisfa il filtro, gli stessi otto campi già presenti in un'unica riga nella tabella denormalizzata `flights_by_status`.

Q2 — 2-3 entità, lookup puntuale

```
1 MATCH (p:Passenger {id: $pid})-[:MAKES]->(r:Reservation)
2     -[:FOR_FLIGHT]->(f:Flight)
3 RETURN r.id, r.reservation_date, r.status, r.total_price
4        ,
5        f.number, f.departure_date, f.arrival_date, f.
6        status
```

Listing 1.7: Q2 Neo4j

```
1 SELECT * FROM reservations_by_passenger
2 WHERE passenger_id = ?
```

Listing 1.8: Q2 Cassandra

Q3 — 4–5 entità, filtro per intervallo

```
1 MATCH (r:Reservation)-[:FOR_FLIGHT]->(f:Flight)
2   -[:OPERATED_BY]->(a:Airline {code: $code})
3 WHERE f.departure_date >= datetime($start)
4   AND f.departure_date <= datetime($end)
5 MATCH (r)-[:FOR_SEAT]->(s:Seat)
6 MATCH (p:Passenger)-[:MAKES]->(r)
7 RETURN r.id, p.name, p.email, f.number, s.number,
8        s.class, r.total_price
```

Listing 1.9: Q3 Neo4j

```
1 SELECT * FROM reservations_by_airline_date
2 WHERE airline_code = ?
3   AND departure_date >= ? AND departure_date <= ?
```

Listing 1.10: Q3 Cassandra

Q4 — Tutte le entità, aggregazione

```
1 MATCH (f:Flight)-
2   [:DEPARTS_FROM]->(ap:Airport {code: $airport})
3 WHERE f.departure_date >= datetime($start)
4   AND f.departure_date <= datetime($end)
5 MATCH (f)-[:OPERATED_BY]->(a:Airline {code: $code})
6 MATCH (r:Reservation)-[:FOR_FLIGHT]->(f)
7 RETURN count(r) AS cnt, sum(r.total_price) AS revenue
```

Listing 1.11: Q4 Neo4j

```
1 SELECT count(*) AS cnt, sum(total_price) AS revenue
2 FROM reservations_by_airline_airport
3 WHERE airline_code = ? AND departure_airport = ?
4   AND departure_date >= ? AND departure_date <= ?
```

Listing 1.12: Q4 Cassandra

1.7.1 Automazione del benchmark

Il benchmark è stato automatizzato in Python. Per ogni dataset e per ogni DBMS, ogni query viene eseguita 31 volte. Il tempo della prima esecuzione viene isolato come misura *cold*; le successive 30 esecuzioni sono mediate e arricchite con la deviazione standard per il calcolo dell'intervallo di confidenza al 95%.

1.8 Esperimenti

1.8.1 Configurazione sperimentale

I test sono stati eseguiti su un laptop equipaggiato con Intel Core i7-12700H (20 thread), 16 GB di RAM e SSD NVMe. Entrambi i DBMS sono stati eseguiti in container Docker con i seguenti limiti:

- Neo4j: heap massimo 2 GB, page cache 1 GB;
- Cassandra: heap massimo 2 GB, new generation 400 MB.

1.9 Risultati

La Tabella 1.1 riassume i tempi della prima esecuzione (cold start) per tutte le configurazioni testate.

Tabella 1.1: Tempi di prima esecuzione (cold) in millisecondi

Dataset	Query	Neo4j (ms)	Cassandra (ms)	Vincitore
25%	Q1	580.79	10.20	Cassandra
25%	Q2	160.14	6.03	Cassandra
25%	Q3	456.88	36.05	Cassandra
25%	Q4	329.59	65.31	Cassandra
50%	Q1	754.67	15.46	Cassandra
50%	Q2	199.74	5.49	Cassandra
50%	Q3	536.84	53.23	Cassandra
50%	Q4	365.23	68.41	Cassandra
75%	Q1	888.75	33.01	Cassandra
75%	Q2	217.00	11.91	Cassandra
75%	Q3	724.64	115.03	Cassandra
75%	Q4	530.35	100.91	Cassandra
100%	Q1	955.03	18.02	Cassandra
100%	Q2	164.29	4.50	Cassandra
100%	Q3	546.75	68.66	Cassandra
100%	Q4	337.20	96.21	Cassandra

Cassandra risulta più veloce di Neo4j in tutte le 16 configurazioni sul tempo di prima esecuzione, con entrambi i container ricreati da zero a ogni

dataset (Sezione 1.11). Cassandra mostra il proprio costo di avvio a freddo (4–115 ms), ma resta comunque sistematicamente più leggero di quello di Neo4j (160–955 ms). La Tabella 1.2 riporta la media delle 30 esecuzioni successive (*warm*) con l’intervallo di confidenza al 95%: è qui, e non nella prima esecuzione, che emerge il quadro più interessante e meno scontato.

Tabella 1.2: Tempi medi a regime (warm, 30 esecuzioni) con IC 95%, in millisecondi

Dataset	Query	Neo4j (ms)	Cassandra (ms)	Vincitore
25%	Q1	106.69 ± 4.46	8.06 ± 0.51	Cassandra
25%	Q2	9.31 ± 1.17	3.35 ± 0.22	Cassandra
25%	Q3	33.88 ± 2.02	13.41 ± 0.88	Cassandra
25%	Q4	10.18 ± 1.23	16.59 ± 1.63	Neo4j
50%	Q1	185.49 ± 8.72	10.99 ± 0.76	Cassandra
50%	Q2	11.08 ± 1.35	4.15 ± 0.37	Cassandra
50%	Q3	69.11 ± 2.99	28.26 ± 3.21	Cassandra
50%	Q4	12.45 ± 0.93	21.06 ± 2.58	Neo4j
75%	Q1	383.55 ± 19.43	22.37 ± 2.32	Cassandra
75%	Q2	16.37 ± 1.41	5.52 ± 0.41	Cassandra
75%	Q3	125.76 ± 7.97	41.22 ± 2.05	Cassandra
75%	Q4	19.28 ± 1.49	36.72 ± 2.55	Neo4j
100%	Q1	335.73 ± 10.84	14.30 ± 2.38	Cassandra
100%	Q2	13.14 ± 1.37	3.23 ± 0.18	Cassandra
100%	Q3	130.59 ± 6.12	44.33 ± 3.47	Cassandra
100%	Q4	14.74 ± 1.10	40.28 ± 4.95	Neo4j

Il quadro sulla media a regime è ora nitido e senza ambiguità statistiche (a differenza di una prima versione dell’esperimento, in cui l’asimmetria tra i due container rendeva Q4 di difficile lettura, riportata in Appendice 1.13 a scopo di confronto): Cassandra vince sempre su Q1, Q2 e Q3, con margine crescente al crescere del dataset; Neo4j vince sempre su Q4, con intervalli di confidenza che non si sovrappongono mai a nessuna delle quattro dimensioni di dataset. Il dettaglio è discusso in Sezione 1.10.

1.10 Analisi dei grafici

Per ogni query sono stati prodotti due istogrammi: uno per la prima esecuzione e uno per la media delle 30 esecuzioni successive, con intervalli di confidenza al 95%. Per i motivi discussi in Sezione 1.11, l'analisi che segue si basa principalmente sulla media a regime, più indicativa del costo intrinseco delle query tra i due DBMS.

Q1 (Figura 1.3): Query su singola entità con filtro per uguaglianza. Il divario è netto e cresce con la dimensione del dataset: la media a regime di Neo4j passa da 106.69 ms (25%) a 335.73 ms (100%, con un picco di 383.55 ms al 75%), mentre Cassandra resta un ordine di grandezza più bassa (8.06–22.37 ms) grazie alla partition key su status. Neo4j deve attraversare tre relazioni aggiuntive (DEPARTS_FROM, ARRIVES_AT, OPERATED_BY) per ogni volo restituito, un costo che la struttura denormalizzata di Cassandra evita per costruzione.

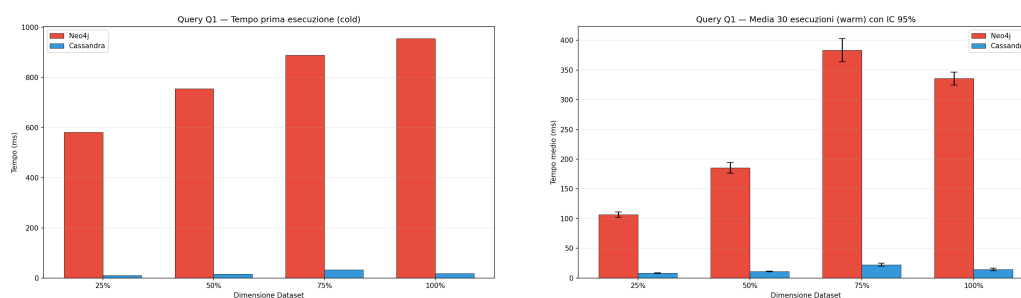


Figura 1.3: Q1 — Voli per stato. Sinistra: prima esecuzione. Destra: media con IC 95%.

Q2 (Figura 1.4): Lookup puntuale su 2–3 entità. Cassandra resta sostanzialmente piatta al crescere del dataset (3.23–5.52 ms): la query filtra per `passenger_id`, chiave di partizione, quindi il suo costo dipende dal numero di prenotazioni di *quel* passeggero, non dalla dimensione totale del dataset. Neo4j cresce un po' di più (9.31–16.37 ms) perché deve comunque attraversare due relazioni (MAKES e FOR_FLIGHT) per ogni prenotazione, ma resta comunque 2–3 volte più lento di Cassandra a ogni dimensione.

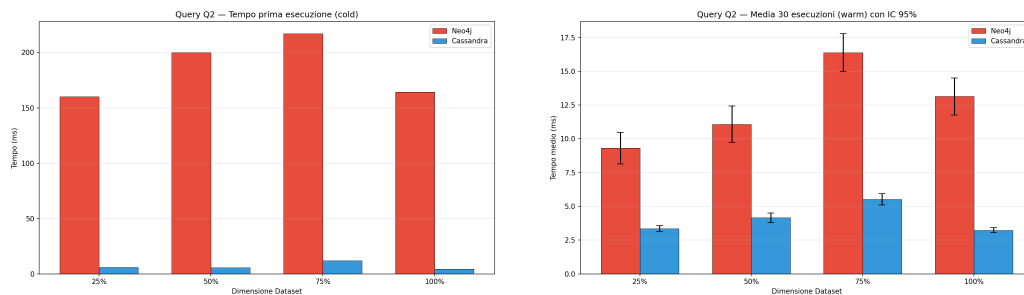


Figura 1.4: Q2 — Prenotazioni di un passeggero. Sinistra: prima esecuzione. Destra: media con IC 95%.

Q3 (Figura 1.5): Query complessa con 4–5 entità e filtro per range. È la query dove il divario cresce più chiaramente con la dimensione del dataset: la media a regime di Neo4j passa da 33.88 ms (25%) a 130.59 ms (100%), un fattore di crescita di circa 3.9×; quella di Cassandra passa da 13.41 ms a 44.33 ms, un fattore molto simile (circa 3.3×). La crescita *relativa* è quindi paragonabile nei due sistemi, ma Neo4j parte da una base assoluta più alta a causa del maggior numero di attraversamenti coinvolti (Reservation→Flight→Airline, più i due MATCH aggiuntivi su Seat e Passenger). Cassandra beneficia della partition key su airline_code e della clustering column su departure_date, che le permettono di scandire la partizione in ordine di data senza dover filtrare l'intero dataset.

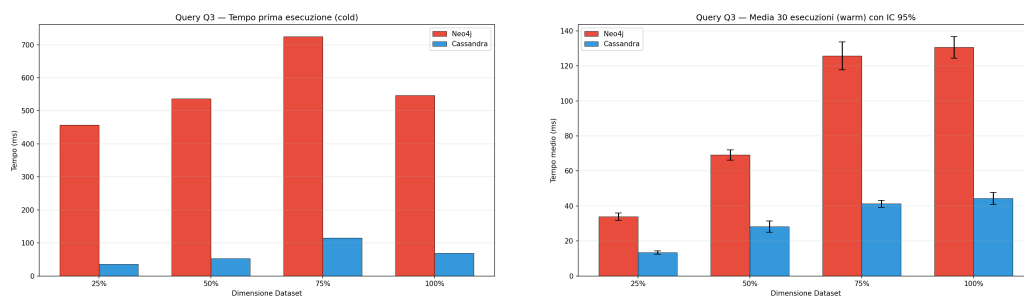


Figura 1.5: Q3 — Prenotazioni per compagnia in intervallo date. Sinistra: prima esecuzione. Destra: media con IC 95%.

Q4 (Figura 1.6): Aggregazione su tutte le entità. È l'unica query dove Neo4j vince, e lo fa in modo netto a tutte e quattro le dimensioni di dataset: 10.18 contro 16.59 ms al 25%, 12.45 contro 21.06 ms al 50%, 19.28 contro 36.72 ms al 75%, 14.74 contro 40.28 ms al 100%. Gli intervalli di confidenza al 95% non si sovrappongono mai, quindi la differenza è statisticamente

significativa a ogni dimensione, non solo sul dataset più grande. La causa è strutturale: Cassandra calcola SUM/COUNT lato server, ma senza un indice pre-aggregato: il coordinator deve comunque scandire sequenzialmente ogni riga della partizione filtrata, un costo che cresce linearmente con il numero di prenotazioni coinvolte (16.59ms→40.28ms, 2.4×). Neo4j, potendo ancorare la query a nodi indicizzati e sommare durante il traversal in memoria, senza l’overhead di lettura/deserializzazione riga per riga dal motore di storage, cresce più lentamente (1.4×).

È l’unico caso, tra le quattro query, in cui il vantaggio nativo del grafo nell’attraversare relazioni supera il costo della denormalizzazione di Cassandra.

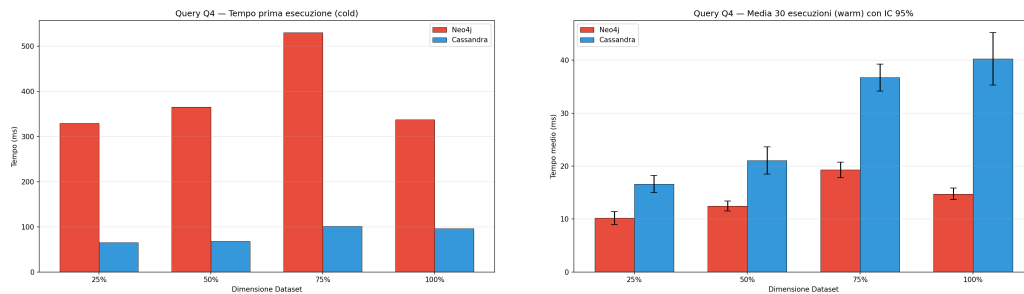


Figura 1.6: Q4 — Aggregazione per aeroporto e compagnia. Sinistra: prima esecuzione. Destra: media con IC 95%.

1.11 Cold vs Warm cache

Un aspetto rilevante è la differenza tra tempo di prima esecuzione e media a regime, osservabile per entrambi i DBMS. In questa versione degli esperimenti, sia il container Neo4j sia il container Cassandra vengono interamente ricreati prima di ogni dataset (Sezione 1.6.2): entrambi i sistemi ripartono quindi da uno stato realmente *freddo* a ogni dataset, il che rende finalmente simmetrico il confronto sui tempi di prima esecuzione tra i due DBMS (Tabella 1.1), superando il limite di una prima versione dell’esperimento in cui solo il container Neo4j veniva ricreato (i cui risultati completi sono riportati in Appendice 1.13, a scopo di trasparenza e confronto).

Anche a parità di condizioni, il rapporto tra prima esecuzione e media a regime resta sistematicamente più alto per Neo4j (da circa 2–5× per Q1 fino a oltre 20–30× per Q2 e Q4, dove il costo a regime è molto basso) rispetto a Cassandra (1.3–4× circa a ogni query e dataset). Questo è coerente con un costo di avvio a freddo, caricamento delle classi, compilazione del piano di

query Cypher, popolamento iniziale della page cache, intrinsecamente più pesante nella JVM di Neo4j rispetto al motore di esecuzione di Cassandra, e non più un artefatto della metodologia sperimentale come nella versione precedente.

Va segnalato, per completezza, che le grandezze assolute misurate in questa esecuzione risultano in generale più alte di quelle registrate in una precedente run dello stesso benchmark, verosimilmente per effetto di condizioni di sistema non perfettamente controllate tra un'esecuzione e l'altra dello script (carico di fondo della macchina, overhead aggiuntivo dei due cicli di ricreazione dei container ora eseguiti a ogni dataset). Per questo motivo le conclusioni si basano sul confronto *relativo* tra i due DBMS all'interno della stessa sessione sperimentale, non sul confronto tra grandezze assolute misurate in esecuzioni diverse dello script.

1.12 Conclusioni

I risultati sperimentali, ottenuti dopo aver corretto la metodologia di generazione dei dataset (veri sottoinsiemi annidati, Sezione 1.6.1), allineato il contenuto informativo restituito da Q1 e Q2 tra i due DBMS (Sezione 1.7), e reso simmetriche le condizioni di *cold start* tra i due container (Sezione 1.11), disegnano un quadro nitido e privo delle ambiguità statistiche emerse nelle versioni precedenti dell'esperimento.

Cassandra vince nettamente su Q1, Q2 e Q3 a ogni dimensione di dataset, grazie alla progettazione *query-first* che denormalizza i dati in tabelle ad-hoc, un vantaggio che cresce con la dimensione del dataset per Q1 (da un fattore 13× al 25% a oltre 23× al 100%) e per Q3, mentre resta pressoché costante per Q2, il cui costo dipende dal numero di prenotazioni del singolo passeggero interrogato più che dalla dimensione totale del dataset.

Neo4j vince altrettanto nettamente su Q4, l'unica query di aggregazione, a tutte e quattro le dimensioni di dataset e con intervalli di confidenza che non si sovrappongono mai. La causa è strutturale: Cassandra non supporta SUM lato server su una partizione distribuita e deve trasferire l'intera partizione al client per aggregarla, un costo che cresce con la dimensione del dataset (quasi 2.4× dal 25% al 100%); Neo4j, potendo ancorare la query a nodi Airport e Airline specifici tramite gli indici univoci e sommare durante il traversal, cresce molto più lentamente (circa 1.4× nello stesso intervallo).

In sintesi:

- per **lookup puntuali e query di selezione su chiavi note in anticipo** (Q1, Q2, Q3), Cassandra è nettamente preferibile, con un vantaggio che si amplifica al crescere del dataset per le query più complesse;
- per **aggregazioni su partizioni che possono diventare molto grandi** (Q4), Neo4j è preferibile: la possibilità di ancorare la query a un nodo indicizzato e aggregare durante il traversal batte sistematicamente l'approccio di Cassandra, che deve trasferire l'intera partizione lato client;
- per **query esplorative non pianificate in anticipo**, il vantaggio strutturale di Neo4j (nessuna necessità di ridisegnare lo schema per ogni nuovo pattern di accesso) resta valido a prescindere dai tempi misurati in questo esperimento specifico.

Il progetto ha inoltre evidenziato l'importanza della metodologia sperimentale ben oltre l'automazione delle 31 esecuzioni: garantire che il contenuto informativo restituito dalle query sia realmente equivalente tra i due DBMS (Sezione 1.7), che i dataset di dimensione crescente siano veri sottoinsiemi annidati e non popolazioni indipendenti (Sezione 1.6.1), e rendere simmetriche le condizioni di cold start tra i due container (Sezione 1.11) sono tutti aspetti che, se ignorati, avrebbero prodotto conclusioni fuorvianti pur partendo dagli stessi dati grezzi.

1.13 Appendice: Risultati con container Cassandra non ricreato

Per trasparenza metodologica, si riportano qui i risultati di una precedente esecuzione dello stesso benchmark, in cui solo il container Neo4j veniva ricreato a ogni dataset mentre Cassandra restava lo stesso processo per l'intera sessione sperimentale (Sezione 1.11). Il confronto tra le Tabelle 1.3 e 1.4 qui sotto e le Tabelle 1.1 e 1.2 nel corpo del report rende concreto l'effetto dell'asimmetria: sui tempi di prima esecuzione il quadro qualitativo non cambia (Cassandra vince comunque tutte le configurazioni), ma sulla media a regime la Q4 risultava qui statisticamente ambigua, indistinguibile ai primi tre dataset, vinta da Cassandra solo al 100%, mentre nella versione corretta dell'esperimento (Tabella 1.2) Neo4j vince Q4 in modo netto e significativo a ogni dimensione.

Tabella 1.3: [Versione precedente, Cassandra non ricreato] Tempi di prima esecuzione (cold) in millisecondi

Dataset	Query	Neo4j (ms)	Cassandra (ms)	Vincitore
25%	Q1	522.05	7.10	Cassandra
25%	Q2	155.96	3.23	Cassandra
25%	Q3	402.85	11.35	Cassandra
25%	Q4	289.34	7.52	Cassandra
50%	Q1	581.03	7.58	Cassandra
50%	Q2	127.93	2.34	Cassandra
50%	Q3	398.81	17.85	Cassandra
50%	Q4	272.61	11.47	Cassandra
75%	Q1	637.82	11.06	Cassandra
75%	Q2	126.88	7.64	Cassandra
75%	Q3	436.47	44.25	Cassandra
75%	Q4	285.70	12.15	Cassandra
100%	Q1	705.20	11.65	Cassandra
100%	Q2	138.93	2.87	Cassandra
100%	Q3	473.56	36.25	Cassandra
100%	Q4	303.80	40.50	Cassandra

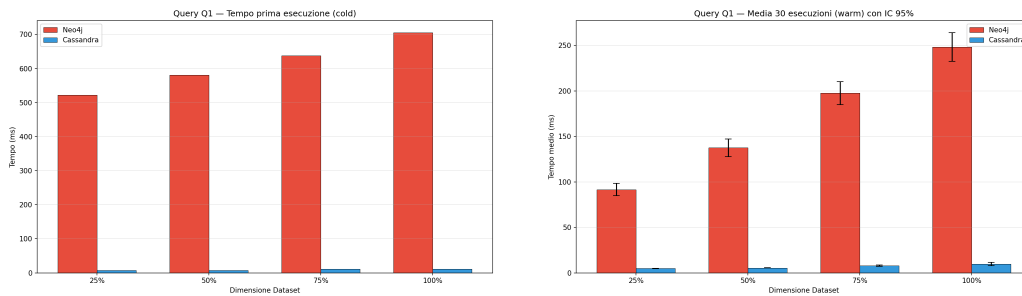


Figura 1.7: Q1 — Voli per stato. Sinistra: prima esecuzione. Destra: media con IC 95%.

Tabella 1.4: [Versione precedente, Cassandra non ricreato] Tempi medi a regime (warm) con IC 95%, in millisecondi

Dataset	Query	Neo4j (ms)	Cassandra (ms)	Vincitore
25%	Q1	91.47 ± 6.80	4.84 ± 0.22	Cassandra
25%	Q2	7.41 ± 0.92	2.23 ± 0.15	Cassandra
25%	Q3	28.48 ± 1.79	9.77 ± 0.54	Cassandra
25%	Q4	8.39 ± 1.02	7.09 ± 0.41	Cassandra
50%	Q1	137.44 ± 9.55	5.52 ± 0.29	Cassandra
50%	Q2	6.85 ± 0.94	2.15 ± 0.26	Cassandra
50%	Q3	48.90 ± 2.85	14.65 ± 0.71	Cassandra
50%	Q4	10.05 ± 0.99	10.88 ± 0.40	Neo4j
75%	Q1	197.59 ± 12.53	7.87 ± 0.65	Cassandra
75%	Q2	9.28 ± 0.78	1.89 ± 0.07	Cassandra
75%	Q3	61.51 ± 3.54	18.13 ± 0.52	Cassandra
75%	Q4	10.85 ± 0.90	10.47 ± 0.77	Cassandra*
100%	Q1	248.25 ± 15.77	9.79 ± 1.73	Cassandra
100%	Q2	9.88 ± 0.83	2.13 ± 0.09	Cassandra
100%	Q3	88.65 ± 3.30	29.41 ± 3.28	Cassandra
100%	Q4	12.28 ± 1.23	24.36 ± 2.74	Neo4j

*differenza entro l'intervallo di confidenza, non statisticamente significativa

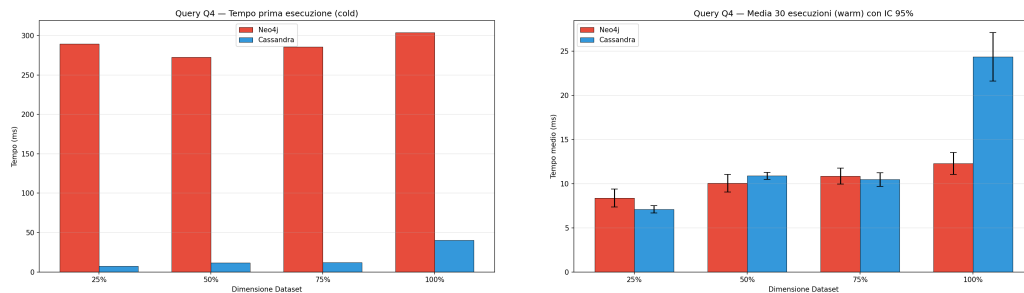


Figura 1.8: Q4 — Aggregazione per aeroporto e compagnia. Sinistra: prima esecuzione. Destra: media con IC 95%.